

AUTHORED BY : ARPIT RAJ , LNMIIT JAIPUR

SELECT Basics

SELECT is used to retrieve data from one or more tables.

General syntax:

```
SELECT column_list  
FROM table_name;
```

SELECT *

The simplest query:

```
SELECT *  
FROM Students;
```

* means:

Return every column.

Internal Working

Execution begins with

```
FROM Students
```

MySQL locates the table.

Then

```
SELECT *
```

returns every column for every row.

Suppose a table has

- 80 columns
- 20 million rows

Using

```
SELECT *
```

reads unnecessary data.

Instead,

```
SELECT Name  
FROM Students;
```

reads only one column.

Less I/O.

Better performance.

Aliases (AS)

Aliases give temporary names to columns or tables.

Syntax

```
SELECT column AS alias  
FROM table;
```

Example

```
SELECT Name AS Student_Name  
FROM Students;
```

Output

Student_Name
Aadya
Arpit

The original column name is not changed.

Multiple aliases

```
SELECT
Name AS Student_Name,
Age AS Student_Age,
Department AS Branch
FROM Students;
```

Table Alias

Suppose

```
SELECT s.Name
FROM Students AS s;
```

We'll use table aliases heavily in JOINS.

DISTINCT

Suppose

Department
ECE
ECE
CSE
ECE
ME

Query

```
SELECT DISTINCT Department
FROM Students;
```

Output

Department
ECE
CSE
ME

Duplicates are removed.

Multiple columns

```
SELECT DISTINCT Department, Gender
FROM Students;
```

Uniqueness is checked on the combination of both columns.

Internal Working

MySQL compares rows and removes duplicate combinations before returning the result.

DISTINCT often requires sorting or hashing internally.

Don't use it unless you actually need unique results.

WHERE

Probably the most frequently used SQL clause.

WHERE filters rows.

Syntax

```
SELECT *
FROM Students
WHERE Age > 20;
```

Only matching rows are returned.

Query

```
SELECT Name
FROM Students
WHERE Age >= 20;
```

Output

Name
Aadya
Arpit

Comparison Operators

Operator	Meaning
=	Equal
!=	Not equal (MySQL)
<>	Not equal (ANSI SQL)

Logical Operators

AND

Both conditions must be true.

```
SELECT *  
FROM Students  
WHERE Department = 'ECE'  
AND CGPA > 8.5;
```

OR

At least one condition must be true.

```
SELECT *  
FROM Students  
WHERE Department = 'ECE'  
OR Department = 'CSE';
```

NOT

Negates a condition.

```
SELECT *  
FROM Students  
WHERE NOT Department = 'ECE';
```

Operator precedence:

1. NOT
2. AND
3. OR

Use parentheses when combining multiple conditions.

```
SELECT *  
FROM Students  
WHERE (Department='ECE' OR Department='CSE')  
AND CGPA > 8;
```

BETWEEN

Checks whether a value lies within an inclusive range.

```
SELECT *  
FROM Students  
WHERE Age BETWEEN 18 AND 22;
```

Equivalent to:

```
WHERE Age >= 18  
AND Age <= 22;
```

Works with:

- Numbers
- Dates
- Strings (lexicographically)

Dates:

```
SELECT *  
FROM Orders  
WHERE OrderDate  
BETWEEN '2025-01-01'  
AND '2025-12-31';
```

IN

Instead of writing:

```
WHERE Department='ECE'  
OR Department='CSE'  
OR Department='IT'
```

Write:

```
SELECT *  
FROM Students  
WHERE Department IN  
( 'ECE', 'CSE', 'IT' );
```

NOT IN

Opposite of IN.

```
SELECT *  
FROM Students  
WHERE Department NOT IN  
( 'ECE', 'CSE' );
```

LIKE

Used for pattern matching.

Two wildcards:

- %
- _

%

Matches zero or more characters.

Starts with A

```
SELECT *  
FROM Students  
WHERE Name LIKE 'A%';
```

Ends with a

```
WHERE Name LIKE '%a';
```

Contains "ad"

```
WHERE Name LIKE '%ad%';
```

—

Matches exactly one character.

```
WHERE Name LIKE '_a%';
```

Examples:

Raj Another:

```
WHERE Name LIKE 'A_____';
```

Matches names starting with A and having exactly five characters. example: Aadya

Wildcards

Wildcard	Meaning
%	Zero or more characters
—	Exactly one character.

IS NULL

Never compare NULL with =.

Wrong:

```
WHERE Department = NULL;
```

Correct:


```
SELECT *  
FROM Students  
WHERE Department IS NULL;
```

IS NOT NULL

```
SELECT *  
FROM Students  
WHERE Email IS NOT NULL;
```

Returns rows where Email has a value.

ORDER BY

Sorts the result.

Ascending (default)

```
SELECT *  
FROM Students  
ORDER BY Age;
```

Explicit:

```
ORDER BY Age ASC;
```

Descending:

```
ORDER BY CGPA DESC;
```

Multiple columns:

```
SELECT *  
FROM Students  
ORDER BY Department ASC,  
CGPA DESC;
```

First sorts by Department.

Within each department,
sorts by CGPA descending.

LIMIT (MySQL)

Returns only the first N rows.

```
SELECT *  
FROM Students  
LIMIT 5;
```

OFFSET

Skip rows.

```
SELECT *  
FROM Students  
LIMIT 5 OFFSET 10;
```

Meaning:

Skip first 10 rows,
return next 5.

Useful for pagination , FETCH FIRST (Standard SQL)

ANSI SQL syntax:

```
SELECT *  
FROM Students  
FETCH FIRST 5 ROWS ONLY;
```

Supported by databases like PostgreSQL, Oracle (12c+), and DB2.

TOP (SQL Server)

SQL Server uses:

```
SELECT TOP 5 *  
FROM Students;
```

Row Limiting Comparison

Database	Syntax
MySQL	LIMIT 5
PostgreSQL	LIMIT 5
SQL Server	TOP 5
ANSI SQL	FETCH FIRST 5 ROWS ONLY

Expressions in SELECT

Arithmetic

```
SELECT
Name,
CGPA + 0.5 AS UpdatedCGPA
FROM Students;
```

String concatenation (MySQL)

```
SELECT
CONCAT(Name, ' - ', Department)
AS StudentInfo
FROM Students;
```

Functions

```
SELECT
UPPER(Name),
LOWER(Name),
LENGTH(Name)
FROM Students;
```

We'll study functions in depth later.

CASE Statement

SQL's equivalent of if-else.

```
SELECT
Name,
CASE
    WHEN CGPA >= 9 THEN 'Excellent'
    WHEN CGPA >= 8 THEN 'Good'
    ELSE 'Average'
END AS Grade
FROM Students;
```

Comments

Single line

```
-- This is a comment
SELECT * FROM Students;
```

MySQL also supports:

```
# This is a comment
```

Multi-line

```
/*
Multiple
line
comment
*/
SELECT * FROM Students;
```

Execution Order of SELECT

This is an extremely common interview question.

Although we write:

```
SELECT Name
FROM Students
WHERE CGPA > 8
ORDER BY Name;
```

SQL logically processes it as:

1. FROM

2. WHERE
3. GROUP BY
4. HAVING
5. SELECT
6. ORDER BY
7. LIMIT

Notice:

SELECT is not executed first, even though it appears first in the query.

This execution order becomes crucial when you start using GROUP BY, aggregates, and aliases.